Java OOP Concepts Program

Java Lab Record

1. Aim

To develop a Java program to demonstrate important Object-Oriented Programming concepts such as Encapsulation, Inheritance, Method Overloading, Method Overriding, Abstraction, Interface, and Polymorphism.

2. Step-by-Step Procedure

1. Create a class named Book with private data members id, name, author, and price.

2. Create a parameterized constructor to initialize the Book object.

3. Create getter and setter methods to access and modify the private data members, demonstrating encapsulation.

4. Create a Person class containing name, age, and a display() method.

5. Create Student and Faculty classes that extend Person, demonstrating inheritance.

6. Override the display() method in Student and Faculty and use super.display() to call the parent class method.

7. Create an Area class with two area() methods having different parameters, demonstrating method overloading.

8. Create a Vehicle class and derive Car and Bike from it.

9. Override the display() method in Car and Bike, demonstrating method overriding.

10. Create an abstract Shape class with an abstract draw() method.

11. Create Circle and Rectangle classes that extend Shape and implement draw(), demonstrating abstraction.

12. Create an interface Printable containing the print() method.

13. Create a Report class that implements Printable.

14. In the main() method, create objects and call the appropriate methods.

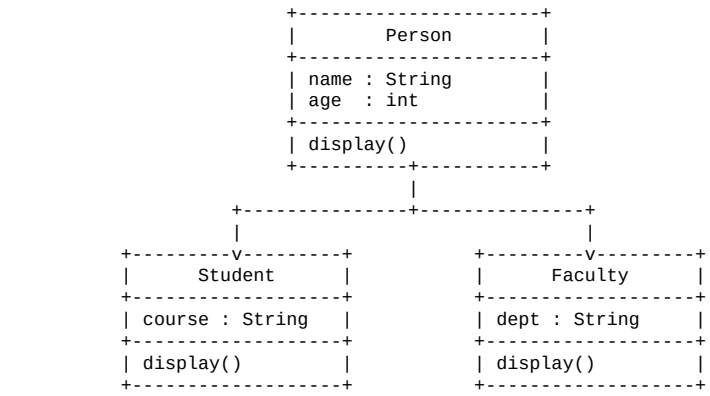
15. Use parent-class references such as Vehicle v1 = new Car() to demonstrate runtime polymorphism.

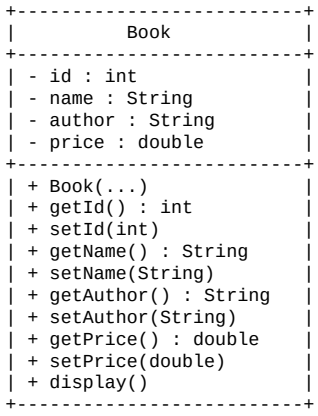
16. Execute the program and observe the output.

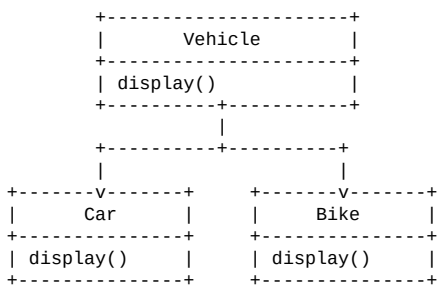
3. Algorithm

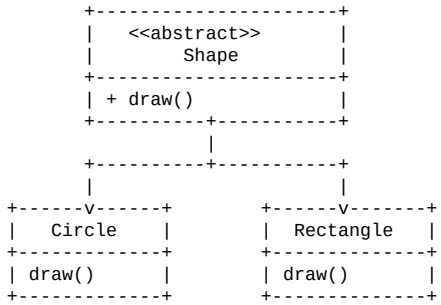
1. Start.
2. Define the Book class with private data members.
3. Initialize the Book object using a constructor.
4. Use getter and setter methods to access the Book data.
5. Define the Person class with name, age, and display().
6. Derive Student and Faculty from Person.
7. Override display() in both child classes.
8. Define the Area class with overloaded area() methods.
9. Define Vehicle and derive Car and Bike.
10. Override display() in Car and Bike.
11. Define the abstract class Shape with the abstract method draw().
12. Derive Circle and Rectangle from Shape and implement draw().
13. Define the Printable interface with the print() method.
14. Implement Printable in the Report class.
15. In main(), create objects of all required classes.
16. Call their respective methods.
17. Display the results.
18. Stop.

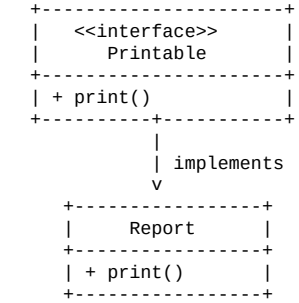
4. UML Class Diagram











5. Code

```
class Book {
    private int id;
    private String name, author;
    private double price;

    Book(int id, String name, String author, double price) {
        this.id = id;
        this.name = name;
        this.author = author;
        this.price = price;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }

    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }

    void display() {
        System.out.println(id + " " + name + " " + author + " " + price);
    }
}

class Person {
    String name;
    int age;

    void display() {
        System.out.println(name + " " + age);
    }
}

class Student extends Person {
    String course;

    void display() {
        super.display();
        System.out.println(course);
    }
}

class Faculty extends Person {
    String dept;

    void display() {
        super.display();
        System.out.println(dept);
    }
}

class Area {
    void area(double r) {
        System.out.println(3.14 * r * r);
    }

    void area(int l, int b) {
        System.out.println(l * b);
    }
}

class Vehicle {
    void display() {
        System.out.println("Vehicle");
    }
}

class Car extends Vehicle {
    void display() {
        System.out.println("Car");
    }
}

class Bike extends Vehicle {
    void display() {
        System.out.println("Bike");
    }
}

abstract class Shape {
```

```
    abstract void draw();
}

class Circle extends Shape {
    void draw() {
        System.out.println("Circle");
    }
}

class Rectangle extends Shape {
    void draw() {
        System.out.println("Rectangle");
    }
}

interface Printable {
    void print();
}

class Report implements Printable {
    public void print() {
        System.out.println("Report");
    }
}

public class Library {
    public static void main(String[] args) {

        Book b = new Book(101, "Java", "Gosling", 500);
        b.display();

        Student s = new Student();
        s.name = "Bob";
        s.age = 20;
        s.course = "Engineering";
        s.display();

        Faculty f = new Faculty();
        f.name = "Dr.Bob";
        f.age = 45;
        f.dept = "CSE";
        f.display();

        Area a = new Area();
        a.area(5.0);
        a.area(10, 20);

        Vehicle v1 = new Car();
        Vehicle v2 = new Bike();
        v1.display();
        v2.display();

        Shape sh1 = new Circle();
        Shape sh2 = new Rectangle();
        sh1.draw();
        sh2.draw();

        Printable p = new Report();
        p.print();
    }
}
```

6. Sample Output

The following output is obtained by executing the given Library program. The program creates objects and displays the results of encapsulation, inheritance, method overloading, overriding, abstraction, interface implementation, and polymorphism.

```
--- Library Program ---
101 Java Gosling 500.0
Bob 20
Engineering
Dr.Bob 45
CSE
78.5
200
Car
Bike
Circle
Rectangle
Report
```